



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Restaurant Management system**” is submitted by

Matta Pravallika - 25B11CS585

Kinthada Akshaya-25B11CS431

in partial fulfillment of the requirements for the award of the B.Tech. degree in
Computer Science and Engineering for the academic year 2026-2027

Project Guide

Mrs. K Vydehi, M.Tech.,(Ph.D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the **Cornerstone project entitled “Restaurant Management System”** is original and is submitted to **ADITYA UNIVERSITY, Surampalem**, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the **Institutional Academic Integrity and AI Usage Policy**.

Place: Surampalem

Date:25-09-2026

BY:

Matta Pravallika - 25B11CS585

Kinthada Akshaya-25B11CS431

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K. Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of our **Restaurant Management System** project.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr. Hari Jyothula, Associate Professor, and Dr. Pennada Siva Satya Prasad, Assistant Professor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M. V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M. B. Srinivas, Vice-Chancellor; Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this **Cornerstone project** successfully.

TABLE OF CONTENTS

1. Cover Page
2. Certificate
3. Acknowledgement
4. Declaration
5. Table of Contents
6. Abstract
7. List of Figures
8. List of Tables
9. Acronyms
10. Introduction
11. System Analysis
12. Database Design
13. ER Diagram
14. Relational Schema
15. Table Design
16. Normalization
17. SQL Implementation
18. SQL Queries & Screenshots
19. Conclusion
20. Future Enhancements
21. References

ABSTRACT

The **Restaurant Management System** is a database application designed to efficiently manage the major operations of a restaurant through a centralized relational database. The system stores and manages information related to employees, managers, waiters, chefs, customers, customer locations, menu items, restaurant orders, payments, and order items.

The system uses primary keys, foreign keys, unique constraints, NOT NULL constraints, and CHECK constraints to maintain data integrity and consistency. The database organizes related information into separate tables, thereby reducing data redundancy and preventing insertion, deletion, and update anomalies.

The database is normalized up to **Third Normal Form (3NF)**. The system supports efficient retrieval of restaurant information through SQL queries involving selection, filtering, joins, aggregation, subqueries, sorting, and reporting.

The proposed Restaurant Management System provides a structured database foundation for managing restaurant employees, customer information, menu items, orders, payments, and order details. It can be further extended into a complete restaurant application with billing, inventory, table reservation, and reporting modules.

LIST OF FIGURES

No.	Title	Page No.
Figure 1	ER Diagram of Restaurant Management System	14
Figure 2	Proposed Three-Layer Database Application Architecture	15
Figure 3	Employee Management Screen	16
Figure 4	Customer Management Screen	17
Figure 5	Menu Management Screen	18
Figure 6	Order and Payment Management Screen	19

LIST OF TABLES

No.	Title	Page No.
Table 1	Major Database Entities	12
Table 2	Employee Table Structure	15
Table 3	Manager Table Structure	16
Table 4	Waiter Table Structure	16
Table 5	Chef Table Structure	16
Table 6	Customer Table Structure	17
Table 7	Menu Table Structure	17
Table 8	Restaurant Orders Table Structure	18
Table 9	Payment Table Structure	18
Table 10	Order Item Table Structure	18
Table 11	Keys and Constraints	19
Table 12	Sample Data Records	20

ACRONYMS

DBMS	- Database Management System
ER	- Entity Relationship
SQL	- Structured Query Language
DDL	- Data Definition Language
DML	- Data Manipulation Language
DQL	- Data Query Language
DCL	- Data Control Language
TCL	- Transaction Control Language
PK	- Primary Key
FK	- Foreign Key
1NF	- First Normal Form
2NF	- Second Normal Form
3NF	- Third Normal Form
UPI	- Unified Payments Interface

1. INTRODUCTION

1.1 Brief Information about the Project:

The **Restaurant Management System** is a database application designed to manage and organize the major activities of a restaurant using a relational database. The system maintains information about employees, customers, menu items, restaurant orders, order items, and payments in a structured manner.

The system is implemented using **Oracle Database** and uses relational database concepts such as primary keys, foreign keys, constraints, joins, aggregate functions, subqueries, and normalization.

The database is designed to reduce data redundancy and improve data consistency by dividing information into appropriate relations. Customer information is separated into CUSTOMER_NEW and CUSTOMER_LOCATION, while employee information is divided into EMPLOYEE, MANAGER, WAITER, and CHEF according to their roles and specializations.

1.2 Motivation and Contribution of Project:

A restaurant handles different types of information every day, including customer details, menu items, employee information, orders, order items, and payments. Maintaining these details manually or in unstructured files can result in duplicate information, inconsistent records, and difficulty in retrieving required information.

The motivation for developing the Restaurant Management System is to provide a structured database that can store restaurant information systematically. The use of a relational database allows related information to be connected through keys and retrieved using SQL queries.

1.3 Objectives of the Project:

The main objectives are:

- To design a relational database for restaurant management.
- To store employee and employee specialization details.
- To maintain customer information and location details.
- To manage menu items, categories, and prices.
- To record restaurant orders and their corresponding customers.
- To maintain individual order items and quantities.
- To record payment information for orders.
- To reduce data redundancy through normalization.
- To maintain data integrity using primary and foreign keys.

1.4 Scope of the Project:

The scope of the Restaurant Management System includes:

- Employee management
- Manager management
- Waiter management
- Chef management
- Customer management
- Menu management
- Order management
- Payment management
- Order-item management
- Revenue analysis
- Customer order analysis
- Menu analysis
- Employee and salary analysis

2. SYSTEM ANALYSIS

2.1 Existing System:

In a basic/manual restaurant system, information may be maintained using notebooks, separate files, spreadsheets, or disconnected records.

Common problems include:

- Manual record maintenance.
- Duplicate customer information.
- Difficulty searching old orders.
- Difficulty tracking payments.
- Difficulty analyzing sales.
- Difficulty managing employee information.
- Higher possibility of data-entry errors.
- Difficulty generating consolidated reports.
- Lack of proper relationships between records.

2.2 Proposed System:

The proposed Restaurant Management System uses an Oracle relational database to organize restaurant information into related tables.

The system consists of nine tables:

1. Employee
2. Manager
3. Waiter
4. Chef
5. Customer
6. Menu
7. Restaurant Orders
8. Payment
9. Order Item

Primary keys uniquely identify records, while foreign keys establish relationships between tables.

The proposed system supports:

- Employee management
- Customer management
- Menu management
- Order management
- Payment management
- Order-item management
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements:

Module	Functional Requirement
Employee Management	Add, update, search and display employee details
Manager Management	Maintain manager bonus information
Waiter Management	Maintain waiter shift information
Chef Management	Maintain chef specialization
Customer Management	Store and retrieve customer information
Menu Management	Maintain food items, categories and prices
Order Management	Store customer orders
Payment Management	Record payments associated with orders
Order Item Management	Store menu items and quantities belonging to orders
Reporting	Generate restaurant reports using SQL queries

2.4 Non-Functional Requirements:

Performance:

The database should provide efficient query execution for normal restaurant workloads.

Integrity:

Primary keys and foreign keys should prevent invalid references and duplicate primary-key values.

Security:

Database privileges should be controlled so that only authorized users can modify important restaurant records.

Reliability:

Transactions should maintain a consistent state of the database.

Scalability:

The database should be capable of handling an increasing number of customers, orders, employees, menu items and payments.

Usability:

The database structure and reports should be understandable to restaurant administrators.

2.5 Requirements Specification

Category	Requirement
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	Adequate storage for Oracle Database
Operating System	Windows 10/11 or equivalent
Database	Oracle Database
Query Language	SQL
Development Environment	Oracle SQL Developer / SQL*Plus

3. DATABASE DESIGN

3.1 Major Entities:

Entity	Description
Employee	Stores common employee information such as employee ID, name, role, phone, salary, and supervisor.
Manager	Stores manager-specific information such as bonus.
Waiter	Stores waiter-specific information such as shift time
Chef	Stores chef-specific information such as specialization
Customer	The normalized customer design is divided into CUSTOMER_NEW and CUSTOMER_LOCATION.
Menu	Stores menu item information including item name, category, and price.
Restaurant Orders	Stores information about orders placed by customers.
Payment	Stores payment information associated with restaurant orders.
Order Item	Stores individual menu items and their quantities within an order.

3.2 ER Model:

The ER model of the Restaurant Management System contains nine major entities.

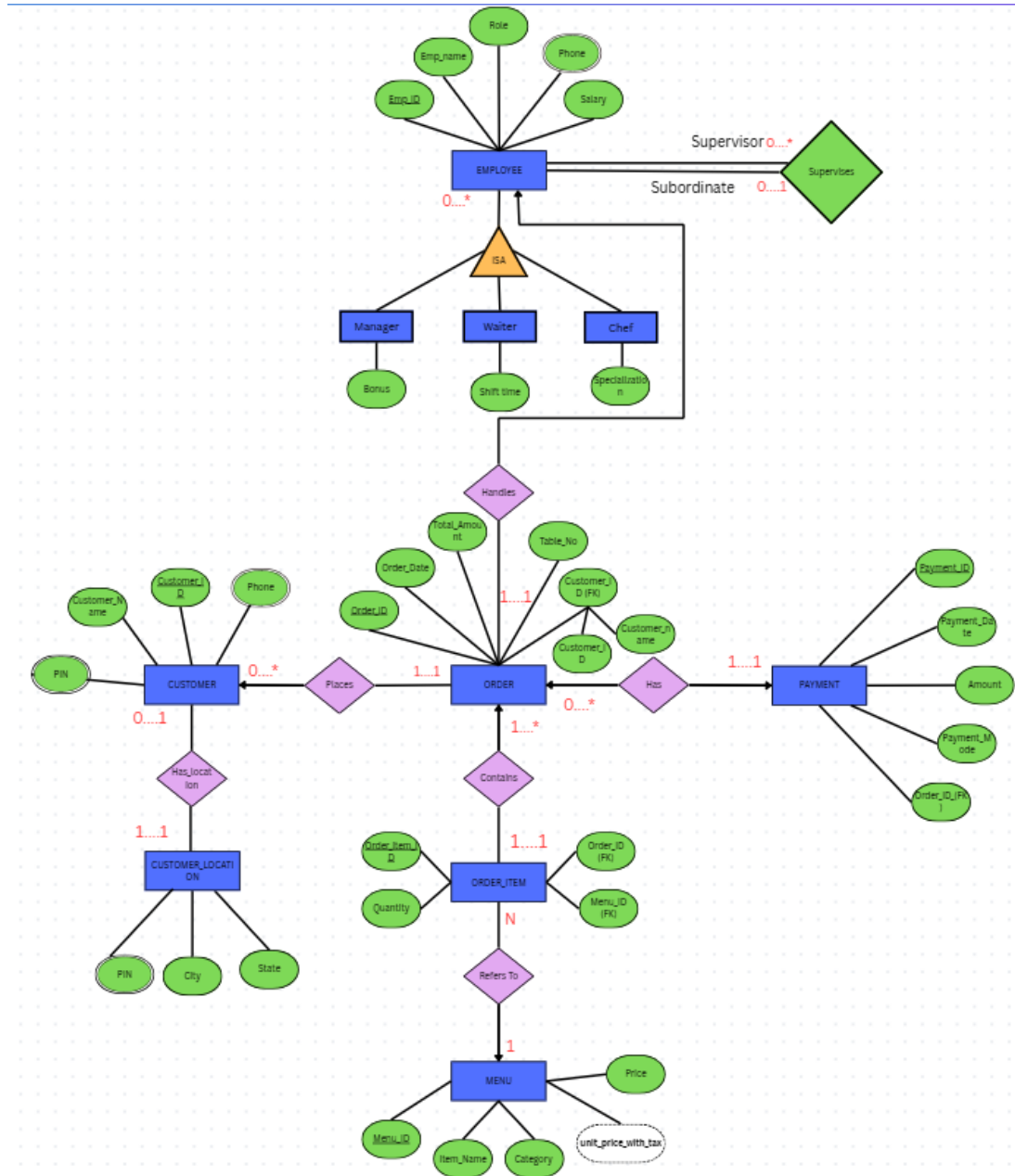
The major relationships are:

- Employee supervises Employee.
- Employee has Manager information.
- Employee has Waiter information.
- Employee has Chef information.
- Customer places Restaurant Orders.
- Restaurant Order has Payment.
- Restaurant Order contains Order Items.
- Menu Item appears in Order Items.

3.3 Relationships:

Relationship	Description
Employee → Employee	An employee may supervise other employees
Employee → Manager	Manager is a specialized employee
Employee → Waiter	Waiter is a specialized employee
Employee → Chef	Chef is a specialized employee
Customer → Restaurant Orders	One customer can place multiple orders
Restaurant Orders → Payment	Payments are associated with orders
Restaurant Orders → Order Item	One order can contain multiple order items
Menu → Order Item	One menu item can occur in multiple order items
Restaurant Orders ↔ Menu	Many-to-many relationship resolved through Order Item

4. ER DIAGRAM



5. RELATIONAL SCHEMA

RELATION	RELATIONAL SCHEMA
1.EMPLOYEE	employee_id(PK), emp_name, role, phone UNIQUE, salary, supervisor_id(FK)
2. MANAGER	employee_id (PK)(FK), bonus
3. WAITER	employee_id (PK)(FK), shift_time
4. CHEF	employee_id (PK)(FK), specialization
5.CUSTOMERS	customer_id, order_id PK, customer_name, phone, pin, city, state, order_date, total_amount
6. CUSTOMER_NEW	customer_id PK, customer_name, phone, pin
7. CUSTOMER_LOCATION	customer_id PK, FK, pin, city, state
8. MENU	menu_id (PK), item_name, category, price
9. RESTAURANT_ORDERS	order_id(PK), order_date, total_amount, table_no, customer_id(FK)
10. PAYMENT	payment_id(PK), payment_date, amount, payment_mode, order_id(FK)
11. ORDER_ITEM	order_item_id(PK), quantity, order_id(FK), menu_id (FK)

6. TABLE DESIGN

6.1 EMPLOYEE Table

Attribute	Data Type	Constraint	Description
employee_id	NUMBER	PK	Unique employee identifier
emp_name	VARCHAR2(100)	NOT NULL	Employee name
role	VARCHAR2(20)	NOT NULL	Employee role
Phone	VARCHAR2(15)	UNIQUE	Employee phone number
salary	NUMBER(10,2)	-	Employee salary
Supervisor_id	NUMBER	FK	Employee's supervisor

6.2 MANAGER Table

Attribute	Data Type	Constraint	Description
employee_id	INTEGER	PK, FK	Manager employee ID
bonus	NUMBER(10,2)	—	Manager bonus

6.3 WAITER TABLE

Attribute	Data Type	Constraint	Description
employee_id	INTEGER	PK, FK	Waiter employee ID
shift_time	VARCHAR2(50)	—	Waiter shift

6.4 CHEF TABLE

Attribute	Data Type	Constraint	Description
employee_id	INTEGER	PK, FK	Chef employee ID
specialization	VARCHAR2(100)	—	Chef specialization

6.5 CUSTOMER_NEW TABLE

Attribute	Data Type	Constraint	Description
customer_id	INTEGER	PK	Unique customer identifier
customer_name	VARCHAR2(100)	NOT NULL	Customer name
phone	VARCHAR2(15)	—	Customer phone
pin	VARCHAR2(10)	—	Postal PIN code

6.6 CUSTOMER_LOCATION

Attribute	Data Type	Constraint	Description
state	VARCHAR2(50)	—	Customer state
city	VARCHAR2(50)	—	Customer city
pin	VARCHAR2(10)	—	Postal PIN code

6.7 MENU TABLE

Attribute	Data Type	Constraint	Description
menu_id	INTEGER	PK	Unique menu item ID
item_name	VARCHAR2(100)	NOT NULL	Name of menu item
category	VARCHAR2(50)	—	Food category
price	NUMBER(10,2)	—	Item price

6.8 RESTAURANT_ORDERS TABLE

Attribute	Data Type	Constraint	Description
order_id	INTEGER	PK	Unique order ID
order_date	DATE	—	Date of order
total_amount	NUMBER(10,2)	—	Total order amount
table_no	NUMBER	—	Restaurant table number
customer_id	NUMBER	NOT NULL, FK	Customer who placed order

6.9 PAYMENT TABLE

Attribute	Data Type	Constraint	Description
payment_id	INTEGER	PK	Unique payment ID
payment_date	DATE	—	Payment date
amount	NUMBER(10,2)	—	Amount paid
payment_mode	VARCHAR2(30)	—	Cash, UPI, Card etc.
order_id	NUMBER	NOT NULL, FK	Associated order

6.10 ORDER_ITEM TABLE

Attribute	Data Type	Constraint	Description
order_item_id	INTEGER	PK	Unique order-item ID
quantity	NUMBER	NOT NULL	Quantity ordered
order_id	NUMBER	NOT NULL, FK	Associated order
menu_id	NUMBER	NOT NULL, FK	Ordered menu item

6.11 KEYS AND CONSTRAINTS

Constraint	Applied To	Purpose
Primary Key	All nine tables	Uniquely identifies each record
Foreign Key	Manager	References Employee
Foreign Key	Waiter	References Employee
Foreign Key	Chef	References Employee
Foreign Key	Employee.supervisor_id	References Employee
Foreign Key	Restaurant Orders	References Customer
Foreign Key	Payment	References Restaurant Orders
Foreign Key	Order Item	References Restaurant Orders
Foreign Key	Order Item	References Menu
Unique	Employee.phone	Prevents duplicate phone numbers
NOT NULL	Required attributes	Prevents missing required values

7. NORMALIZATION

Normalization organizes data into related tables to reduce redundancy and avoid insertion, deletion, and update anomalies. The sample documentation also presents normalization from UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF.

7.1 Unnormalized Form (UNF):

A single restaurant transaction record might contain **Order_ID**, **Order_Date**, **Customer_ID**, **Customer_Name**, **Customer_Phone**, **Table_No**, **multiple Menu_Item values**, **multiple Quantity values**, **Payment_ID**, **Payment_Mode**, and **Payment_Amount** in one record. Repeating menu-item and quantity groups, along with repeated customer and payment information, create **data redundancy and update anomalies**. This unstructured form represents the **Unnormalized Form (UNF)** of the Restaurant Management System.

7.2 First Normal Form (1NF):

In 1NF, repeating groups are removed and each attribute contains atomic values. Each order item is represented as a separate row with its corresponding **Order_ID**, **Menu_ID**, and **Quantity**. Customer, order, menu, and payment details may still be repeated across multiple order-item records. This removes repeating groups but data redundancy may still exist.

7.3 Second Normal Form (2NF):

In 2NF, partial dependencies on a composite key are removed. In the Restaurant Management System, **Order_ID** determines order-specific attributes such as **Order_Date**, **Total_Amount**, **Table_No**, and **Customer_ID**, while **Menu_ID** determines menu-specific attributes such as **Item_Name**, **Category**, and **Price**. Order-item-specific attributes, such as **Quantity**, depend on the particular order item. These dependencies motivate separating the **RESTAURANT_ORDERS**, **MENU**, and **ORDER_ITEM** relations, thereby reducing redundancy and improving data consistency.

7.4 Third Normal Form (3NF):

In 3NF, transitive dependencies are removed by separating attributes that depend on other non-key attributes.

In the Restaurant Management System, customer details are separated into **CUSTOMER_NEW** and **CUSTOMER_LOCATION**. The **CUSTOMER_NEW** table stores customer-specific information such as **Customer_ID**, **Customer_Name**, **Phone**, and **PIN**, while **CUSTOMER_LOCATION** stores location-specific information such as **PIN**, **City**, and **State** and is linked to the customer through **Customer_ID**.

Similarly, menu information is stored separately in MENU, order information in RESTAURANT_ORDERS, payment information in PAYMENT, and order-item information in ORDER_ITEM. Employee-related information is divided among EMPLOYEE, MANAGER, WAITER, and CHEF.

Relation	Reason for Separation
EMPLOYEE	Stores common employee details such as name, role, phone, salary, and supervisor.
MANAGER	Stores manager-specific information such as bonus.
WAITER	Stores waiter-specific information such as shift time.
CHEF	Stores chef-specific information such as specialization.
CUSTOMER_NEW	Stores customer-specific details such as Customer_ID, Customer_Name, Phone, and PIN.
CUSTOMER_LOCATION	Stores location-related details such as PIN, City, and State and references the corresponding customer.
MENU	Stores menu item details such as item name, category, and price.
RESTAURANT_ORDERS	Stores order-specific details and references the customer who placed the order.
PAYMENT	Stores payment details such as payment date, amount, and payment mode and references the order.
ORDER_ITEM	Stores the quantity of menu items ordered and connects each order with its corresponding menu item.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

Employee Table

```
CREATE TABLE employee (  
    employee_id NUMBER PRIMARY KEY,  
    emp_name VARCHAR2(100) NOT NULL,  
    role VARCHAR2(20) NOT NULL,  
    phone VARCHAR2(15) UNIQUE,  
    salary NUMBER(10,2),  
    supervisor_id NUMBER,  
  
    CONSTRAINT fk_employee_supervisor  
    FOREIGN KEY (supervisor_id)  
    REFERENCES employee(employee_id)  
);
```

Manager table

```
CREATE TABLE manager (  
    employee_id INTEGER PRIMARY KEY,  
    bonus NUMBER(10,2),  
  
    CONSTRAINT fk_manager_employee  
    FOREIGN KEY (employee_id)  
    REFERENCES employee(employee_id)  
);
```

Waiter table

```
CREATE TABLE waiter (  
    employee_id INTEGER PRIMARY KEY,  
    shift_time VARCHAR2(50),
```

```

CONSTRAINT fk_waiter_employee
FOREIGN KEY (employee_id)
REFERENCES employee(employee_id)
);

```

Chef

```

CREATE TABLE chef (
    employee_id INTEGER PRIMARY KEY,
    specialization VARCHAR2(100),

    CONSTRAINT fk_chef_employee
    FOREIGN KEY (employee_id)
    REFERENCES employee(employee_id)
);

```

Customer Normalized Tables:

Customer_new table

```

CREATE TABLE customer_new (
    customer_id NUMBER
        CONSTRAINT pk_customer_new PRIMARY KEY,

    customer_name VARCHAR2(50)
        CONSTRAINT nn_customer_name NOT NULL,

    phone VARCHAR2(15)
        CONSTRAINT nn_customer_phone NOT NULL
        CONSTRAINT uq_customer_phone UNIQUE,

    pin VARCHAR2(10)
        CONSTRAINT nn_customer_pin NOT NULL
);

```

Customer_location table

```
CREATE TABLE customer_location (  
    customer_id NUMBER  
    CONSTRAINT pk_customer_location PRIMARY KEY,  
    pin VARCHAR2(10)  
    CONSTRAINT nn_location_pin NOT NULL,  
    city VARCHAR2(50)  
    CONSTRAINT nn_location_city NOT NULL,  
    state VARCHAR2(50)  
    CONSTRAINT nn_location_state NOT NULL,  
    CONSTRAINT fk_location_customer  
    FOREIGN KEY (customer_id)  
    REFERENCES customer_new(customer_id)  
);
```

Menu Table

```
CREATE TABLE menu (  
    menu_id INTEGER PRIMARY KEY,  
    item_name VARCHAR2(100) NOT NULL,  
    category VARCHAR2(50),  
    price NUMBER(10,2)  
);
```

Restaurant Orders

```
CREATE TABLE restaurant_orders (  
    order_id INTEGER PRIMARY KEY,  
    order_date DATE,  
    total_amount NUMBER(10,2),  
    table_no NUMBER,  
    customer_id NUMBER NOT NULL,
```

```
CONSTRAINT fk_order_customer  
FOREIGN KEY (customer_id)  
REFERENCES customer(customer_id)  
);
```

Payment

```
CREATE TABLE payment (  
    payment_id INTEGER PRIMARY KEY,  
    payment_date DATE,  
    amount NUMBER(10,2),  
    payment_mode VARCHAR2(30),  
    order_id NUMBER NOT NULL,  
  
    CONSTRAINT fk_payment_order  
    FOREIGN KEY (order_id)  
    REFERENCES restaurant_orders(order_id)  
);
```

Order Item

```
CREATE TABLE order_item (  
    order_item_id INTEGER PRIMARY KEY,  
    quantity NUMBER NOT NULL,  
    order_id NUMBER NOT NULL,  
    menu_id NUMBER NOT NULL,  
  
    CONSTRAINT fk_orderitem_order  
    FOREIGN KEY (order_id)  
    REFERENCES restaurant_orders(order_id),  
  
    CONSTRAINT fk_orderitem_menu  
    FOREIGN KEY (menu_id)
```

```
REFERENCES menu(menu_id)
);
```

8.2 DML – Data Manipulation Language

The database is populated using INSERT statements.

For example:

```
INSERT INTO employee (employee_id, emp_name, role, phone, salary,
supervisor_id)
VALUES (1, 'Amit', 'Manager', '90000000001', 60000, NULL);
```

8.3 DQL – Data Query Language

```
1. SELECT MIN(salary) AS minimum_salary,
      MAX(salary) AS maximum_salary
      FROM employee;
```

```
2. SELECT role,
      SUM(salary) AS total_salary
      FROM employee
      GROUP BY role;
```

```
3. SELECT employee_id,
      emp_name,
      role,
      salary
      FROM employee
      WHERE emp_name LIKE 'A%';
```

```

4. select emp_name from Employee
   where salary > 50000;

5. select e.employee_id,e.emp_name,e.role,e.salary,m.bonus
   from Employee e
   join manager m
   on e.employee_id = m.employee_id;

```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

1. Minimum and Maximum Salary

```

SELECT MIN(salary) AS minimum_salary,
       MAX(salary) AS maximum_salary
FROM employee;

```

Output:

MINIMUM_SALARY	MAXIMUM_SALARY
24000	62000

2. Total Salary by Role

```

SELECT role,
       SUM(salary) AS total_salary
FROM employee
GROUP BY role;

```

Output:

ROLE	TOTAL_SALARY
Manager	885000
Waiter	374000
Chef	700000

3. Employees Whose Name Starts With A

```
SELECT employee_id,emp_name,role,salary  
FROM employee  
WHERE emp_name LIKE 'A%';
```

Output:

EMPLOYEE_ID	EMP_NAME	ROLE	SALARY
1	Amit	Manager	60000
5	Arjun	Manager	61000
10	Anjali	Manager	61000
18	Ajay	Waiter	26000
23	Aakash	Waiter	24000
30	Aman	Waiter	26000

4. Employees Earning More Than ₹50,000

```
select emp_name from Employee  
where salary > 50000;
```

Output:

EMP_NAME
Amit
Rahul
Priya
Neha
Arjun
Riya
Vikram
Sneha
Karan
Anjali
Rohan
Pooja
Suresh
Meena
Deepak

5. Employee and Manager Bonus Details

```
select e.employee_id,e.emp_name,e.role,e.salary,m.bonus
```

```
select e.employee_id, e.emp_name, e.role, e.salary, m.bonus
```

```
from Employee e
```

```
join manager m
```

```
on e.employee_id = m.employee_id;
```

Output:

EMPLOYEE_ID	EMP_NAME	ROLE	SALARY	BONUS
1	Amit	Manager	60000	10000
2	Rahul	Manager	58000	9000
3	Priya	Manager	62000	11000
4	Neha	Manager	59000	9500
5	Arjun	Manager	61000	10000
6	Riya	Manager	57000	8500
7	Vikram	Manager	60000	9000
8	Sneha	Manager	56000	8000
9	Karan	Manager	59000	9500
10	Anjali	Manager	61000	10000
11	Rohan	Manager	58000	9000
12	Pooja	Manager	57000	8500
13	Suresh	Manager	62000	11000
14	Meena	Manager	55000	8000
15	Deepak	Manager	60000	10000

6. Orders With Customer Details

```

select o.order_id,o.order_date,c.customer_name,o.total_amount,o.table_no
from restaurant_orders o
join customer c
on o.customer_id = c.customer_id;

```

Output:

ORDER_ID	ORDER_DATE	CUSTOMER_NAME	TOTAL_AMOUNT	TABLE_NO
1	01-SEP-26	John	350	1
2	01-SEP-26	Sarah	280	2
3	02-SEP-26	David	220	3
4	03-SEP-26	John	250	4
5	03-SEP-26	Sarah	180	5
6	04-SEP-26	Anita	150	6
7	04-SEP-26	Rahul	200	7
8	05-SEP-26	David	190	8
9	05-SEP-26	Priya	120	9
10	06-SEP-26	Aman	320	10
11	06-SEP-26	John	200	11
12	07-SEP-26	Neha	300	12
13	07-SEP-26	Vikram	180	13
14	08-SEP-26	Sneha	260	14
15	08-SEP-26	Sarah	160	15

7. Customers With More Than One Visit

```
select c.customer_id,c.customer_name,count(o.order_id) as no_of_visits
from customer c
join restaurant_orders o
on c.customer_id = o.customer_id
GROUP BY c.customer_id,c.customer_name
HAVING COUNT(o.order_id) > 1;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME	NO_OF_VISITS
1	John	3
2	Sarah	3
3	David	2

8. Total Restaurant Revenue

```
select sum(amount) as total_revenue
```

from payment;

Output:

TOTAL_REVENUE
3360

9. Total Number of Orders

```
select count(*) as total_orders
```

```
from restaurant_orders;
```

Output:

TOTAL_ORDERS
15

10. Payment Details With Customer Information

```
SELECT
```

```
    p.payment_id,
```

```
    c.customer_name,
```

```
    p.amount,
```

```
    p.payment_mode,
```

```
    p.payment_date
```

```
FROM payment p
```

```
JOIN restaurant_orders o
```

```
ON p.order_id = o.order_id
```

```
JOIN customer c
```

```
ON o.customer_id = c.customer_id;
```

Output:

PAYMENT_ID	CUSTOMER_NAME	AMOUNT	PAYMENT_MODE	PAYMENT_DATE
1	John	350	Cash	01-SEP-26
2	Sarah	280	UPI	01-SEP-26
3	David	220	Card	02-SEP-26
4	John	250	Cash	03-SEP-26
5	Sarah	180	UPI	03-SEP-26
6	Anita	150	Card	04-SEP-26
7	Rahul	200	Cash	04-SEP-26
8	David	190	UPI	05-SEP-26
9	Priya	120	Card	05-SEP-26
10	Aman	320	Cash	06-SEP-26
11	John	200	UPI	06-SEP-26
12	Neha	300	Card	07-SEP-26
13	Vikram	180	Cash	07-SEP-26
14	Sneha	260	UPI	08-SEP-26
15	Sarah	160	Card	08-SEP-26

11. Revenue by Payment Mode

```
select payment_mode,sum(amount) as total_revenue
from payment
group by payment_mode;
```

Output:

PAYMENT_MODE	TOTAL_REVENUE
Card	950
Cash	1300
UPI	1110

12. Most Expensive Menu Item

```
select * from menu
where price = (
    select max(price) from menu
);
```

Output:

MENU_ID	ITEM_NAME	CATEGORY	PRICE
1	Margherita Pizza	Pizza	350

13. Menu Items Included in Orders

```
select m.menu_id,m.item_name,m.category,m.price,o.order_item_id,o.quantity
from menu m
join order_item o
on o.menu_id = m.menu_id;
```

Output:

MENU_ID	ITEM_NAME	CATEGORY	PRICE	ORDER_ITEM_ID	QUANTITY
1	Margherita Pizza	Pizza	350	1	1
2	Chicken Biryani	Main Course	280	2	1
3	Veg Biryani	Main Course	220	3	1
4	Paneer Butter Masala	Main Course	250	4	1
5	Chicken Burger	Fast Food	180	5	1
6	Veg Burger	Fast Food	150	6	1
7	Hakka Noodles	Chinese	200	7	1
8	Fried Rice	Chinese	190	8	1
9	Masala Dosa	South Indian	120	9	1
10	Butter Chicken	Main Course	320	10	1
11	French Fries	Snacks	100	11	2
12	Chocolate Cake	Dessert	150	12	2
13	Ice Cream	Dessert	90	13	2
14	Cold Coffee	Beverage	130	14	2
15	Fresh Lime Soda	Beverage	80	15	2

14. Quantity Sold for Each Menu Item

```
SELECT m.menu_id,
       m.item_name,
       SUM(oi.quantity) AS total_quantity_sold
FROM menu m
JOIN order_item oi
```

```

ON m.menu_id = oi.menu_id

GROUP BY m.menu_id, m.item_name

ORDER BY total_quantity_sold DESC;

```

Output:

MENU_ID	ITEM_NAME	TOTAL_QUANTITY_SOLD
14	Cold Coffee	2
12	Chocolate Cake	2
13	Ice Cream	2
15	Fresh Lime Soda	2
11	French Fries	2
3	Veg Biryani	1
5	Chicken Burger	1
10	Butter Chicken	1
7	Hakka Noodles	1
4	Paneer Butter Masala	1
1	Margherita Pizza	1
9	Masala Dosa	1
6	Veg Burger	1
8	Fried Rice	1
2	Chicken Biryani	1

15. Customers Who Have Never Placed an Order

```

SELECT c.customer_id,
       c.customer_name
FROM customer c
LEFT JOIN restaurant_orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

```

Output:

CUSTOMER_ID	CUSTOMER_NAME
11	Rohan
12	Pooja
13	Karan
14	Meera
15	Arjun

16. Number of Orders Placed by Each Customer

```

SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS no_of_orders
FROM customer c
LEFT JOIN restaurant_orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY no_of_orders DESC;

```

Output:

CUSTOMER_ID	CUSTOMER_NAME	NO_OF_ORDERS
1	John	3
2	Sarah	3
3	David	2
10	Sneha	1
8	Neha	1
7	Aman	1
5	Rahul	1
6	Priya	1
4	Anita	1
9	Vikram	1
14	Meera	0
12	Pooja	0
13	Karan	0
15	Arjun	0
11	Rohan	0

17. Orders Above Average Order Amount

```
SELECT order_id,  
       order_date,  
       total_amount,  
       table_no,  
       customer_id  
FROM restaurant_orders  
WHERE total_amount > (  
    SELECT AVG(total_amount)  
    FROM restaurant_orders  
);
```

Output:

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	TABLE_NO	CUSTOMER_ID
1	01-SEP-26	350	1	1
2	01-SEP-26	280	2	2
4	03-SEP-26	250	4	1
10	06-SEP-26	320	10	7
12	07-SEP-26	300	12	8
14	08-SEP-26	260	14	10

18. Orders With Payment Information

```
SELECT o.order_id,  
       o.order_date,  
       o.total_amount,  
       p.payment_mode,  
       p.amount  
FROM restaurant_orders o  
JOIN payment p  
ON o.order_id = p.order_id;
```

Output:

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	PAYMENT_MODE	AMOUNT
1	01-SEP-26	350	Cash	350
2	01-SEP-26	280	UPI	280
3	02-SEP-26	220	Card	220
4	03-SEP-26	250	Cash	250
5	03-SEP-26	180	UPI	180
6	04-SEP-26	150	Card	150
7	04-SEP-26	200	Cash	200
8	05-SEP-26	190	UPI	190
9	05-SEP-26	120	Card	120
10	06-SEP-26	320	Cash	320
11	06-SEP-26	200	UPI	200
12	07-SEP-26	300	Card	300
13	07-SEP-26	180	Cash	180
14	08-SEP-26	260	UPI	260
15	08-SEP-26	160	Card	160

19. Customers Who Paid Using UPI

```
SELECT DISTINCT c.customer_id,  
               c.customer_name  
FROM customer c  
JOIN restaurant_orders o  
ON c.customer_id = o.customer_id  
JOIN payment p  
ON o.order_id = p.order_id  
WHERE p.payment_mode = 'UPI';
```

Output:

CUSTOMER_ID	CUSTOMER_NAME
1	John
2	Sarah
3	David
10	Sneha

20. Total Manager Bonus

```
SELECT SUM(bonus) AS total_manager_bonus  
FROM manager;
```

Output:

TOTAL_MANAGER_BONUS
141000

21. Managers With Bonus Greater Than ₹9,000

```
SELECT e.employee_id,  
       e.emp_name,  
       m.bonus  
FROM employee e  
JOIN manager m  
ON e.employee_id = m.employee_id  
WHERE m.bonus > 9000;
```

Output:

EMPLOYEE_ID	EMP_NAME	BONUS
1	Amit	10000
3	Priya	11000
4	Neha	9500
5	Arjun	10000
9	Karan	9500
10	Anjali	10000
13	Suresh	11000
15	Deepak	10000

22. Find employees working as Chefs

```
SELECT employee_id,  
       emp_name,  
       phone,  
       salary
```

FROM employee

WHERE role = 'Chef';

Output:

EMPLOYEE_ID	EMP_NAME	PHONE	SALARY
31	Chef Raj	9000000031	45000
32	Chef Anand	9000000032	47000
33	Chef Mohan	9000000033	46000
34	Chef Arvind	9000000034	48000
35	Chef Kiran	9000000035	45000
36	Chef Rakesh	9000000036	47000
37	Chef Sohan	9000000037	46000
38	Chef Naveen	9000000038	49000
39	Chef Dinesh	9000000039	45000
40	Chef Mahesh	9000000040	48000
41	Chef Vinod	9000000041	47000
42	Chef Suraj	9000000042	46000
43	Chef Raju	9000000043	49000
44	Chef Imran	9000000044	45000
45	Chef Sameer	9000000045	47000

23. Find the highest-paid employee

SELECT employee_id,

emp_name,

role,

salary

FROM employee

WHERE salary = (

SELECT MAX(salary)

FROM employee

);

Output:

EMPLOYEE_ID	EMP_NAME	ROLE	SALARY
3	Priya	Manager	62000
13	Suresh	Manager	62000

24. Find the average employee salary

```
SELECT AVG(salary) AS average_salary
FROM employee;
```

Output:

AVERAGE_SALARY
43533.3333333333333333333333333333

25. Find orders placed by a particular customer

```
SELECT order_id,
       order_date,
       total_amount,
       table_no
FROM restaurant_orders
WHERE customer_id = 1;
```

Output:

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	TABLE_NO
1	01-SEP-26	350	1
4	03-SEP-26	250	4
11	06-SEP-26	200	11

26. Find the most expensive order

```
SELECT order_id,
       order_date,
       total_amount,
       table_no,
       customer_id
FROM restaurant_orders
```

```
WHERE total_amount = (
    SELECT MAX(total_amount)
    FROM restaurant_orders
);
```

Output:

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	TABLE_NO	CUSTOMER_ID
1	01-SEP-26	350	1	1

27. Find the number of employees in each role

```
SELECT role,
    COUNT(*) AS number_of_employees
FROM employee
GROUP BY role;
```

Output:

ROLE	NUMBER_OF_EMPLOYEES
Manager	15
Waiter	15
Chef	15

28. Find the total quantity ordered for each order

```
SELECT order_id,
    SUM(quantity) AS total_quantity
FROM order_item
GROUP BY order_id;
```

Output:

ORDER_ID	TOTAL_QUANTITY
1	1
6	1
11	2
13	2
2	1
14	2
4	1
5	1
8	1
3	1
7	1
9	1
10	1
12	2
15	2

29. Display customers and their orders

```
SELECT c.customer_name,  
       o.order_id,  
       o.order_date,  
       o.total_amount  
FROM customer_new c  
JOIN restaurant_orders o  
ON c.customer_id = o.customer_id;
```

Output:

CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
John	1	01-SEP-26	350
Sarah	2	01-SEP-26	280
David	3	02-SEP-26	220
John	4	03-SEP-26	250
Sarah	5	03-SEP-26	180
Anita	6	04-SEP-26	150
Rahul	7	04-SEP-26	200
David	8	05-SEP-26	190
Priya	9	05-SEP-26	120
Aman	10	06-SEP-26	320
John	11	06-SEP-26	200
Neha	12	07-SEP-26	300
Vikram	13	07-SEP-26	180
Sneha	14	08-SEP-26	260
Sarah	15	08-SEP-26	160

30. Find the total revenue generated from each customer's orders

```
SELECT c.customer_id,  
       c.customer_name,  
       SUM(o.total_amount) AS total_spent  
FROM customer_new c  
JOIN restaurant_orders o  
ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENT
2	Sarah	620
5	Rahul	200
3	David	410
8	Neha	300
10	Sneha	260
1	John	800
9	Vikram	180
4	Anita	150
6	Priya	120
7	Aman	320

10. CONCLUSION

The **Restaurant Management System** provides a structured relational database for managing important restaurant operations. The system organizes employee, customer, menu, order, order-item, and payment information into separate relations.

The use of primary keys and foreign keys establishes relationships between the tables and helps maintain data integrity. The normalization process from **UNF to 1NF, 2NF, and 3NF** reduces redundancy and improves consistency.

The separation of customer information into **CUSTOMER_NEW** and **CUSTOMER_LOCATION** provides a normalized approach for maintaining customer and location information independently. Similarly, separate relations for **MENU**, **RESTAURANT_ORDERS**, **ORDER_ITEM**, and **PAYMENT** allow restaurant transactions to be represented clearly.

SQL queries demonstrate the ability to retrieve, filter, aggregate, and combine information from different relations. Overall, the project demonstrates the practical application of relational database concepts using Oracle Database.

11. FUTURE ENHANCEMENTS

The Restaurant Management System can be extended in several ways:

1. **Table Reservation Module**
A reservation table can be added to manage customer reservations.
2. **Inventory Management**
A separate inventory module can track ingredients and stock levels.
3. **Billing System**
Automatic bill generation can be integrated with orders and payments.
4. **Online Ordering**
Customers could place orders through a web or mobile interface.
5. **Employee Attendance**
Attendance and working-hour records could be added.
6. **Sales Reports**
Daily, weekly, and monthly sales reports can be generated.
7. **Role-Based Access**
Different database privileges can be provided to managers, waiters, and administrators.

8. **Graphical User Interface**

A frontend application can be connected to the Oracle database for easier interaction.

12. REFERENCES

1. **Oracle Database Documentation** – Oracle SQL and relational database concepts.
2. **Database Management System Course Materials** – Concepts related to relational databases, normalization, SQL, keys, and constraints.
3. **Oracle SQL Developer** – Used for database creation, SQL execution, and query testing.
4. **Aditya University DBMS Project Guidelines and Course Material** – Used as the academic basis for the project structure and database concepts.
5. **Project Database Design and SQL Implementation** – Restaurant Management System developed as part of the DBMS Cornerstone Project.

Reference Books:

1. Database Systems, Ramez Elmasri, Shamkant B. Navathe, 7th Edition, Pearson, ISBN: 9780137504277.
2. Database Systems, Carlos Coronel, Steven Morris, Peter Rob, 10th Edition, Cengage, ISBN: 978-0357517864.
3. Introduction to Database Systems, C J Date, Pearson, 9th Edition, ISBN: 978-0133970777.

Textbooks:

1. Database System Concepts ,Abraham Silberschatz, Henry Korth, and S. Sudarshan, McGrawHill , 8th Edition, ISBN-13: 978-1260230508.
2. 2 .Database Management Systems, Raghurama Krishnan, Johannes Gehrke, Mc Graw-Hill, 4th Edition, ISBN: 978-1260091195.